```
 1  using DataDefinitions.Interfaces;
 2  using DataDefinitions.Models;
 3  using LiteDB;
 4  using System;
 5  using System.Collections.Generic;
 6  using System.Collections.ObjectModel;
 7  using System.Diagnostics;
 8  using System.IO;
 9  using System.IO.Pipes;
10  using System.Linq;
11  using System.Text;
12  using System.Threading.Tasks;
13
14  namespace DataDefinitions.LiteDBSupport;
15
16  public class LiteDBDal : ILiteDBDAL, IReferenceUsage<FilamentDefn>,
      IEventListener
17  {
18
19      const string settingTableName = "settings";
20      const string printSettingTableName = "printSettings";
21      const string vendorTableName = "vendors";
22      const string noteTableName = "notes";
23      const string filamentTableName = "filaments";
24      const string inventoryTableName = "inventory";
25
26      //#if DEBUG
27
28      //        static readonly string testingFilename = Path.Combine
          (Environment.GetFolderPath
          (Environment.SpecialFolder.ApplicationData),
          "FilamentDataLiteDb.db");
29      //#else
30      //        static readonly string testingFilename = Path.Combine
          (Environment.GetFolderPath
          (Environment.SpecialFolder.ApplicationData),
          "FilamentDataLiteDb.db");
31      //#endif
32
33
34      public ObservableCollection<FilamentDefn> Filaments
35      {
36          get; protected set;
37      }
38
39      public CollateTagsCollection<VendorDefn> Vendors
40      {
41          get; protected set;
42      }
```

```
43      public CollateTagsCollection<NoteDefn> Notes
44      {
45          get; protected set;
46      }
47
48      public List<Setting> Settings
49      {
50          get; protected set;
51      }
52
53      public ObservableCollection<PrintSettingDefn> PrintSettings
54      {
55          get; protected set;
56      }
57
58      public string Filename
59      {
60          get; protected set;
61      }
62      protected string ConnectionString => $"filename={Filename};
          Connection=\"shared\"";
63      /// <summary>
64      /// Initialize the LiteDb Data Layer
65      /// </summary>
66      /// <param name="filename">filename to be access, currently ignored
          using TestingFilamentDataLDB.db in the My Documents directory</
          param>
67      /// <param name="platform"></param>
68      public void Initialize(string filename, UIPlatform platform =
          UIPlatform.WPF)
69      {
70          //Debug.Assert(filename != null);
71          if (string.IsNullOrEmpty(filename))
72          {
73              Filename = FileLocator.DatabasePath(platform);
74          }
75          else
76              Filename = FileLocator.DatabasePath(platform);
77          //var cString = $"filename={Filename}; Connection=\"shared\"";
78          ConnectionString connection = new ConnectionString
              (ConnectionString);
79          PrepareToDeserialize();
80          using LiteDatabase database = new LiteDatabase(connection);
81          var setting = database.GetCollection<Setting>
              (settingTableName).Query().ToEnumerable().FirstOrDefault(s =>
              s.Name.Contains("SeedData"));
82          if (setting == null || setting < DataSeed.AddInventorySpools)
83              SeedDatabase();
84
```

```
85          Settings = new List<Setting>(database.GetCollection<Setting>
              (settingTableName).Query().ToEnumerable());
86          //PrintSettings = new ObservableCollection<PrintSettingDefn>
              (database.GetCollection<PrintSettingDefn>
              (printSettingTableName).Query().ToEnumerable());
87
88          Filaments = new ObservableCollection<FilamentDefn>
              (database.GetCollection<FilamentDefn>(filamentTableName).Query
              ().ToEnumerable());
89          Vendors = new CollateTagsCollection<VendorDefn>
              (database.GetCollection<VendorDefn>(vendorTableName).Query
              ().ToEnumerable());
90          // populate the Inventory
91
92          foreach (var filament in Filaments)
93          {
94              if(filament.VendorID is int id && id != 0)
95              {
96                  filament.Vendor=Vendors.Where(v=>
                      v.VendorDefnId==id).SingleOrDefault();
97              }
98          }
99
100         foreach (var vendor in Vendors)
101         {
102             vendor.LinkInventoryToFilaments(Filaments);
103             //vendor.LinkChildren(vendor);
104             vendor.PostDataRetrieveActions();
105             //vendor.LinkVendorSettingsToPrintDefns(PrintSettings);
106         }
107         Notes = new CollateTagsCollection<NoteDefn>
              (database.GetCollection<NoteDefn>(noteTableName).Query
              ().ToEnumerable());
108         FinishDeserialization();
109     }
110     public LiteDBDal()
111     {
112         //var entityBuilder = BsonMapper.Global.Entity<SpoolDefn>
              ().DbRef<InventorySpool>(s => s.Inventory,"inventory");
113     }
114     public void SetFilename(string filename) => Filename = filename;
115     public void SmartAdd(DataObject dbObject)
116     {
117         switch (dbObject)
118         {
119             case VendorDefn vendorDefn:
120                 vendorDefn.PrepareToSave(this);
121                 if (vendorDefn.InDatabase)
122                     Update(vendorDefn);
```

```
123                 else
124                 {
125                     Add(vendorDefn);
126                     if (!Vendors.Contains(vendorDefn))
127                         Vendors.Add(vendorDefn);
128                 }
129                 break;
130             case FilamentDefn filamentDefn:
131                 if (filamentDefn.InDatabase)
132                     Update(filamentDefn);
133                 else
134                 {
135                     Add(filamentDefn);
136                     if (!Filaments.Contains(filamentDefn))
137                         Filaments.Add(filamentDefn);
138                 }
139                 break;
140             case Setting setting:
141                 if (setting.InDatabase)
142                     Update(setting);
143                 else
144                 {
145                     Add(setting);
146                     if (!Settings.Contains(setting))
147                         Settings.Add(setting);
148                 }
149                 break;
150 
151             case NoteDefn noteDefn:
152                 if (noteDefn.InDatabase)
153                     Update(noteDefn);
154                 else
155                 {
156                     Add(noteDefn);
157                     if (!Notes.Contains(noteDefn))
158                         Notes.Add(noteDefn);
159                 }
160                 break;
161             default:
162                 throw new NotSupportedException($"Support for
                      {dbObject.GetType().Name} is not implemented");
163         }
164         dbObject.IsModified = false;
165         dbObject.SetContainedModifiedState(false);
166     }
167     public void Upsert(IEnumerable<DatabaseObject> adds)
168     {
169         switch (adds)
170         {
```

```
171
172             case IEnumerable<VendorDefn> vendorDefinitions:
173                 Upsert(vendorDefinitions, vendorTableName);
174                 break;
175             case IEnumerable<FilamentDefn> filamentDefinitions:
176                 Upsert(filamentDefinitions, filamentTableName);
177                 break;
178             default:
179                 Trace.WriteLine($"Upsert not implemented for IEnumerable<
                      {GetType().GetGenericArguments().FirstOrDefault()?.Name}
                      >");
180                 break;
181         }
182
183
184     }
185     private void Upsert<TAdd>(IEnumerable<TAdd> adds, string
          collectionName) where TAdd : DatabaseObject
186     {
187         using LiteDatabase db = new LiteDatabase(ConnectionString);
188         db.GetCollection<TAdd>(collectionName).Upsert(adds);
189
190     }
191     private void Add<TAdd>(TAdd add, string collectionName) where TAdd :
          DatabaseObject
192     {
193         using LiteDatabase database = new LiteDatabase(ConnectionString);
194         database.GetCollection<TAdd>(collectionName).Insert(add);
195     }
196     public void Add(VendorDefn vendor)
197     {
198         Add(vendor, vendorTableName);
199         //using(LiteDatabase database=new LiteDatabase(ConnectionString))
200         //{
201         //    database.GetCollection<VendorDefn>("vendors").Insert
              (vendor);
202         //}
203         //throw new NotImplementedException();
204     }
205
206     public void Add(FilamentDefn filament)
207     {
208         Add(filament, filamentTableName);
209         //throw new NotImplementedException();
210     }
211
212     public void Add(Setting setting)
213     {
214         Add(setting, settingTableName);
```

```
215         //throw new NotImplementedException();
216     }
217
218     public void Add(PrintSettingDefn printSettingDefn)
219     {
220         Add(printSettingDefn, printSettingTableName);
221         //throw new NotImplementedException();
222     }
223     public void Add(NoteDefn noteDefn)
224     {
225         Add(noteDefn, noteTableName);
226     }
227     //public void Add(InventorySpool inventorySpool) => Add
        (inventorySpool, inventoryTableName);
228     private void Update<TUpdate>(TUpdate update, string collectionName)
        where TUpdate : DatabaseObject
229     {
230         using LiteDatabase database = new LiteDatabase(ConnectionString);
231         database.GetCollection<TUpdate>(collectionName).Update(update);
232     }
233     public void Update(VendorDefn vendor)
234     {
235         Update(vendor, vendorTableName);
236         //throw new NotImplementedException();
237     }
238
239     public void Update(FilamentDefn filament)
240     {
241         Update(filament, filamentTableName);
242         //throw new NotImplementedException();
243     }
244
245     public void Update(Setting setting)
246     {
247         Update(setting, settingTableName);
248         //throw new NotImplementedException();
249     }
250
251     public void Update(PrintSettingDefn printSettingDefn)
252     {
253         Update(printSettingDefn, printSettingTableName);
254         //throw new NotImplementedException();
255     }
256     public void Update(NoteDefn noteDefn)
257     {
258         Update(noteDefn, noteTableName);
259     }
260     //public void Update(InventorySpool inventorySpool) => Update
        (inventorySpool, inventoryTableName);
```

```
261     public void Remove(FilamentDefn filament)
262     {
263         Debug.Assert(!filament.IsIntrinsic, "Can't delete intrinsic
              filaments");
264
265         using LiteDatabase database = new LiteDatabase(ConnectionString);
266         database.GetCollection<FilamentDefn>(filamentTableName).Delete
              (filament.FilamentDefnId);
267     }
268     //public void Remove(InventorySpool inventorySpool)
269     //{
270     //    using LiteDatabase db = new LiteDatabase(ConnectionString);
271     //    db.GetCollection<InventorySpool>(inventoryTableName).Delete
            (inventorySpool.KeyID);
272     //}
273     public void SeedDatabase()
274     {
275         if (string.IsNullOrEmpty(Filename))
276             throw new Exception("Filename is not initialized, unable to
                  seed the database.");
277         else
278         {
279             var cString = $"filename={Filename}; Connection=\"shared\"";
280             using LiteDatabase database = new LiteDatabase(cString);
281             Seed(database);
282         }
283         //throw new NotImplementedException();
284     }
285     internal  void Seed(ILiteDatabase liteDatabase)
286     {
287         var seeding = liteDatabase.GetCollection<Setting>
              (settingTableName).Query().ToArray().FirstOrDefault<Setting>(s
              => s.Name.Contains(Constants.dataSeedingKey));
288         if (seeding == null || seeding < DataSeed.InitialSeed)
289         {
290             Debug.WriteLine("Seeding filaments");
291             InitialSeeding(liteDatabase, ref seeding);
292         }
293         if (seeding < DataSeed.AddVendorDefn)
294         {
295             Debug.WriteLine("Seeding vendors");
296             SeedVendorDefn(liteDatabase, seeding);
297         }
298         if (seeding < DataSeed.AddInventorySpools)
299         {
300             Debug.WriteLine("Seeding inventory");
301             SeedInventory(liteDatabase, seeding);
302         }
303
```

```
304     }
305
306     private static void InitialSeeding(ILiteDatabase liteDatabase, ref
          Setting seeding)
307     {
308         var col = liteDatabase.GetCollection<FilamentDefn>
              (filamentTableName);
309         col.InsertBulk(DataDefinitions.Seed.InitialFilamentDefinitions());
310         foreach (var item in col.Query().ToArray())
311         {
312             item.LinkChildren<FilamentDefn>(item);
313             col.Update(item);
314         }
315         if (seeding != null)
316         {
317             seeding.SetValue(DataSeed.InitialSeed);
318             liteDatabase.GetCollection<Setting>(settingTableName).Update
                  (seeding);
319         }
320         else
321         {
322             seeding = new Setting()
323             {
324                 Name = Constants.dataSeedingKey,
325                 Value = DataSeed.InitialSeed.ToString()
326             };
327
328             liteDatabase.GetCollection<Setting>(settingTableName).Insert
                  (seeding);
329         }
330     }
331     private static void SeedVendorDefn(ILiteDatabase liteDatabase,
          Setting seeding)
332     {
333         var col = liteDatabase.GetCollection<VendorDefn>(vendorTableName);
334         var vendors = DataDefinitions.Seed.InitialVendorDefinitions();
335         col.InsertBulk(vendors);
336         foreach (var item in vendors)
337             col.Update(item);
338         seeding.SetValue(DataSeed.AddVendorDefn);
339         liteDatabase.GetCollection<Setting>(settingTableName).Update
              (seeding);
340     }
341     private void SeedInventory(ILiteDatabase liteDatabase, Setting
          seeding)
342     {
343         var col = liteDatabase.GetCollection<VendorDefn>(vendorTableName);
344         var flashforge = col.Query().ToEnumerable().First<VendorDefn>(v
              => v.Name.Contains("Flashforge"));
```

```
345         var plaFilament = liteDatabase.GetCollection<FilamentDefn>
              (filamentTableName).Query().ToEnumerable().First(f =>
              f.MaterialType == MaterialType.PLA);
346         var sd = flashforge.SpoolDefns.First();
347         InventorySpool inventory = new InventorySpool()
348         {
349             ColorName = "Orange",
350             DateOpened = DateTime.Today - TimeSpan.FromDays(5),
351             FilamentDefn = plaFilament,
352             FilamentDefnId = plaFilament.FilamentDefnId,
353             Parent = sd
354             //SpoolDefn = sd,
355             //SpoolDefnId = sd.SpoolDefnId
356         };
357         inventory.LinkToParent(sd);
358         var dm = new DepthMeasurement() { InventorySpool = inventory,
              Depth1 = 32.2, Depth2 = 31.1, MeasureDateTime = DateTime.Now };
359         inventory.DepthMeasurements.Add(dm);
360         sd.Inventory.Add(inventory);
361
362         col.Update(flashforge);
363
364         seeding.SetValue(DataSeed.AddInventorySpools);
365         liteDatabase.GetCollection<Setting>(settingTableName).Update
              (seeding);
366     }
367     private static void SeedPrintSettings(ILiteDatabase database, Setting
        setting)
368     {
369         var col = database.GetCollection<PrintSettingDefn>
              (printSettingTableName);
370
371         col.InsertBulk(DataDefinitions.Seed.InitialPrintSettingDefinitions
              ());
372         setting.SetValue(DataSeed.AddPrinterSettingDefn);
373         database.GetCollection<Setting>(settingTableName).Update(setting);
374     }
375     protected static void PrepareToDeserialize()
376     {
377         FilamentDefn.InDataOps = true;
378         VendorDefn.InDataOps = true;
379         SpoolDefn.InDataOps = true;
380         DensityAlias.InDataOps = true;
381         PrintSettingDefn.InDataOps = true;
382         DepthMeasurement.InDataOps = true;
383         InventorySpool.InDataOps = true;
384         VendorPrintSettingsConfig.InDataOps = true;
385         ConfigItem.InDataOps = true;
386         NoteDefn.InDataOps = true;
```

```
387     }
388     protected static void FinishDeserialization()
389     {
390         FilamentDefn.InDataOps = false;
391         VendorDefn.InDataOps = false;
392         SpoolDefn.InDataOps = false;
393         DensityAlias.InDataOps = false;
394         PrintSettingDefn.InDataOps = false;
395         DepthMeasurement.InDataOps = false;
396         InventorySpool.InDataOps = false;
397         VendorPrintSettingsConfig.InDataOps = false;
398         ConfigItem.InDataOps = false;
399         NoteDefn.InDataOps = false;
400     }
401
402     public IEnumerable<DataObject> GetReferences(FilamentDefn item)
403     {
404         foreach (var vendor in Vendors)
405             foreach (var vendorRef in vendor.GetReferences(item))
406                 yield return vendorRef;
407         //throw new NotImplementedException();
408     }
409     public void ReleaseEventHandlers()
410     {
411         foreach (var vendor in Vendors)
412             vendor.ReleaseEventHandlers();
413         foreach (var filament in Filaments)
414             filament.ReleaseEventHandlers();
415
416     }
417 }
418
```